

Master Upgrading PostgreSQL, In the Real World

Pat Wright

PostgreSQL Advocate

<https://github.com/sqlasylum/presentation>



@sqlasylum



sqlasylum@gmail.com



/in/patwright



sqlasylum.wordpress.com/

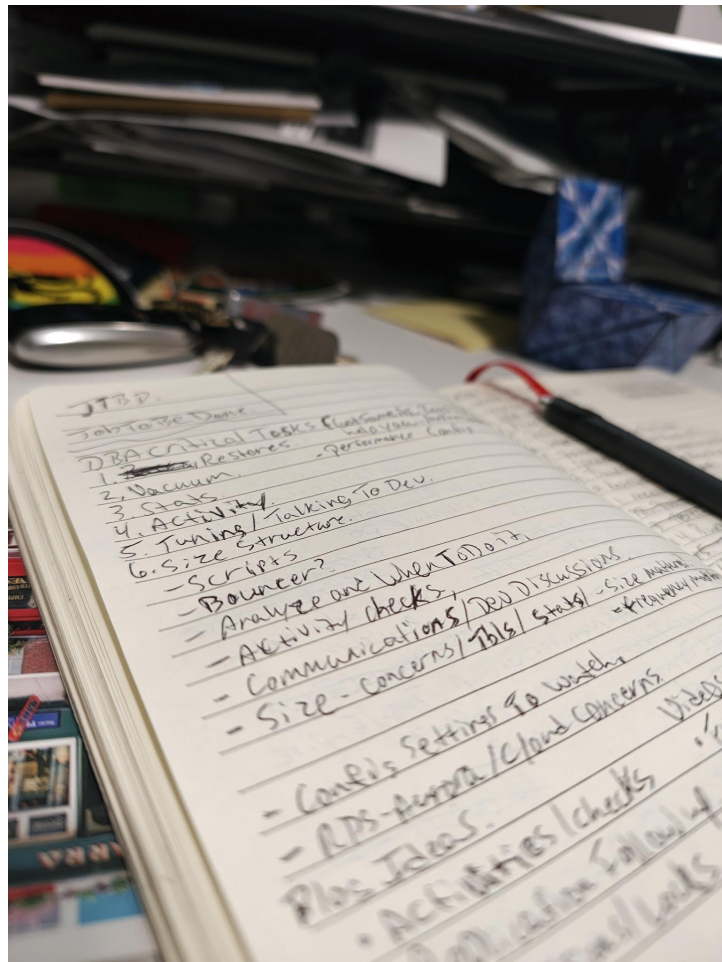


Master Upgrading PostgreSQL. Using Real World stories and examples



Agenda

- Why is this important?
- Lay the groundwork
- PG_Upgrade
- Key Tasks
- Large Scale Upgrades
- Real world fails
- Summary



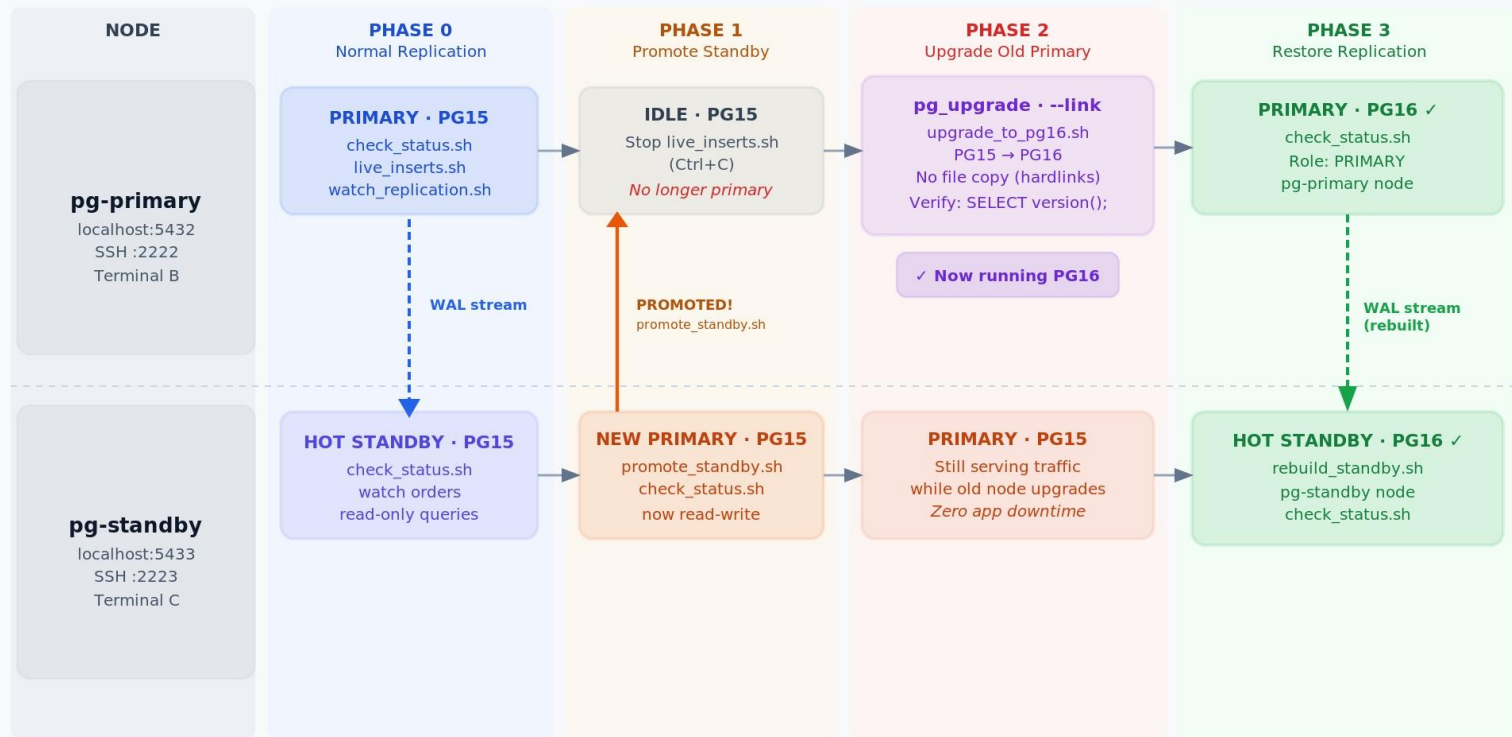
Why Upgrading matters?

- Critical for support
- Critical for features
- Critical for life of application
- It's not easy.



PostgreSQL Upgrade Demo — Promote & Upgrade Pattern

pg-primary (PG15 → PG16) · pg-standby (PG15 → PG16) · Docker · pg_upgrade --link



Key Scripts (demo_script.md)

check_status.sh
Shows version + role
on any node

live_inserts.sh
Continuous demo
data into orders

promote_standby.sh
pg_ctl promote
on standby node

upgrade_to_pg16.sh
pg_upgrade --link
PG15 → PG16

rebuild_standby.sh
pg_basebackup from
new PG16 primary

watch_replication.sh
pg_stat_replication lag
pg_stat_wal_receiver

Promote & Upgrade

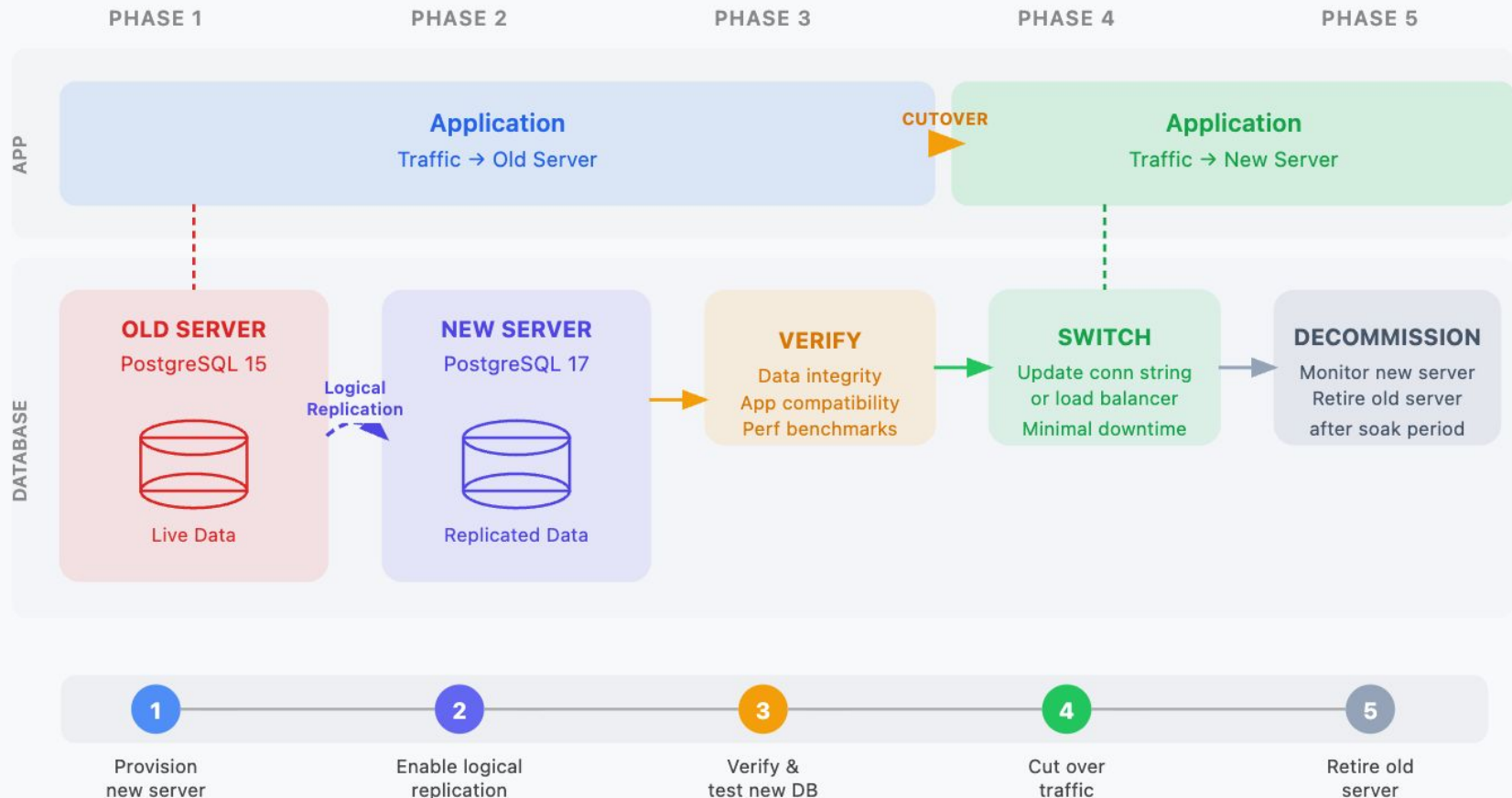
Pros

- Low downtime
- Testing....ish
- Automatic Scripting

Cons

- Lots of Steps
- Secondary setups
- Cost of multiple servers

PG Upgrade Replication/sync



Replication sync

Pros

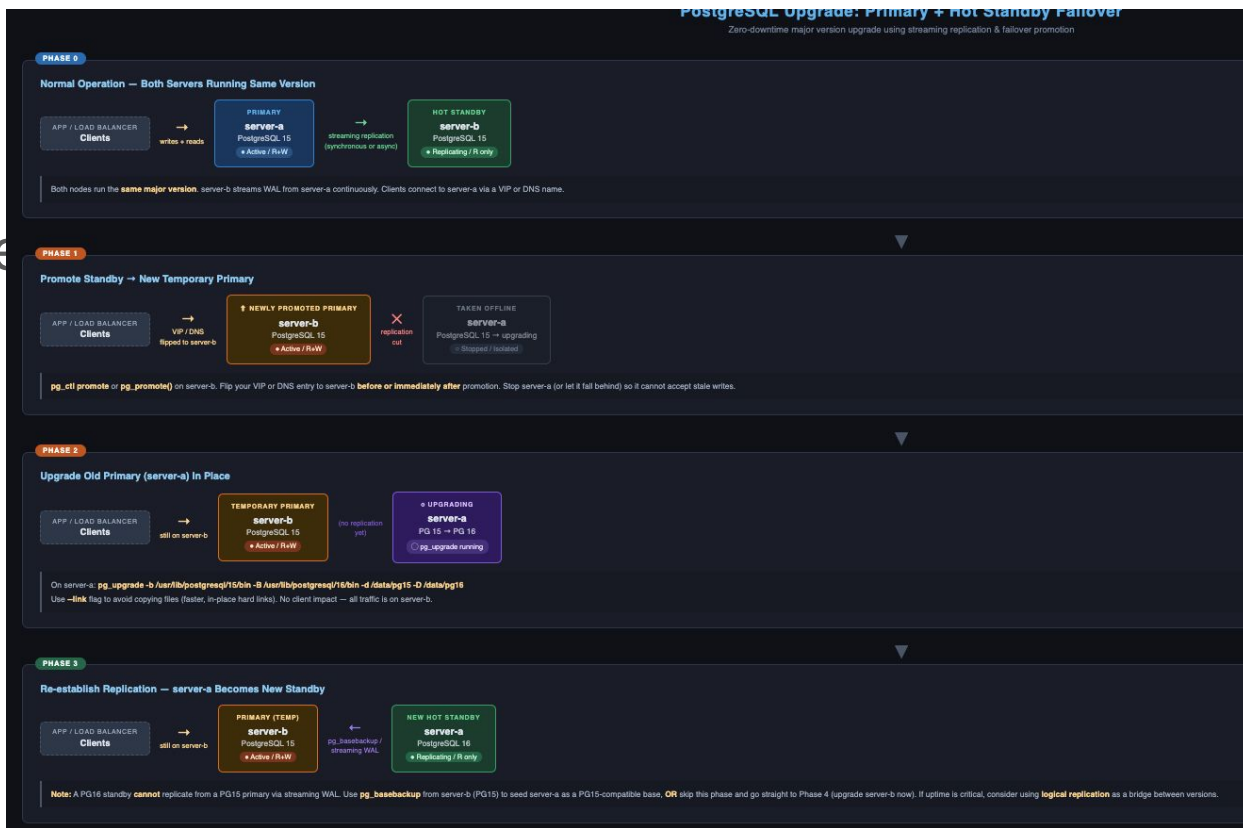
- Very Low Downtime
- Pre-setup
- Testing....ish
- Automatic Scripting

Cons

- Scale issues
- Can impact Production
- Can take a long time
- Can miss things
- Setup

PG_Upgrade

- Demo Process
- Stats still need to be updated
- Timings



Key's for all Upgrade Projects

1. Find what is changing – Release Notes
2. Clone and Test – A lot , Seriously Lots
3. Dev upgrade and testing
4. Check with Teams/Communication
5. Determine Downtime
6. This is NOT the only upgrade



Running your Upgrade Incident

- EVERYONE involved
- Checklist/Task list.
- Go/No Go for Launch
- Point of no Return
- Automation!



Failures

- Pg 15, “the AND issue”

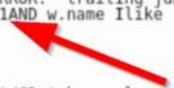
```
pg_query(): Query Failed: ERROR: trailing junk after numeric literal at or near ...
```

After reviewing a sampling of the errors on [this spreadsheet](#), I found that most of the screenshots reviewed show this error.

What happened is that in versions prior to Postgresql v15, this was an allowed action. However, in Postgresql v15, this was “fixed”. This error happens when a numeric literal or UUID is passed into the condition of a query and has another condition directly following (think use of AND/OR after passing in the condition value).

Here is a screenshot of the error.

```
-----
HTTP_X_FORWARDED_FOR: 52.71.222.43, 172.70.135.160
REMOTE_ADDR: 10.81.214.197
Description:
pg_query(): Query failed: ERROR: trailing junk after numeric literal at or near "200281A"
LINE 15:      c.id = 200281AND w.name ilike '%Specials Nine%' OR w.i...
                        ^
Backtrace
-----
/var/www/Entrata/Eos/Connect/CDatabase.class.php:~pq_query
```



This is taken from the Postgresql v15 release notes.

- Adjust **JSON** numeric literal processing to match the SQL/JSON-standard (Peter Eisentraut)

This accepts numeric formats like `.1` and `1.`, and disallows trailing junk after numeric literals, like `1.type()`.

Failures

- The reporting query
- Rolling Forward with Production Code
- Weekend Warrior



Summary

- Planning is important.
- Get Used to Upgrading
- Built in Tools can be great!
- Find the unknowns to have less failures



Pat Wright

PostgreSQL Advocate

<https://github.com/sqlasylum/presentation>



@sqlasylum



sqlasylum@gmail.com



/in/patwright



sqlasylum.wordpress.com/



Master Upgrading PostgreSQL, Using Real World stories and examples

